INFO0010-4 : Project 1 - Tic tac toe

Romain Malaise

20 octobre 2025

1 Software Architecture

The program is divided into 8 classes, each with a unique role :

— **TictactoeServer** : Main class of the server. Its role is to create a server socket that listens for incoming client connections and accepts them. For each new connection, it creates a client socket and starts a new ClientHandler thread.

— **ClientHandler** : handles communication with a single client. Parses commands, sends responses, interacts with the Game object, and manages the connection.

— **Game** : represents the Tic tac toe game and logic. Stores the grid, verifies moves, and determines the game state. Can manage the single and multiplayer mode, by setting or not the two ClientHandler.

— **BotPlayer** : Represents a bot and manages its random moves in single-player mode.

— **Matchmaking** : pairs two clients for multiplayer sessions. (Can be viewed as a waiting room)

— **MessageReader/MessageWriter** : Reading and sending requests, respecting the Tic tac toe protocol. MessageWriter is a small class to avoid redundancy when closing a message according to the protocol. It automatically appends the empty line terminator, writes, and flushes the data.

This structure in classes of the project ensures a good modularity, readability and clarity.

2 Program Logic

When the server starts, it opens a ServerSocket on port 2791 and waits/listens for client connections. For each connection :

1. The server accepts it and creates a dedicated socket. This socket is paired with the client's socket, forming the TCP channel used for communication.
2. A new ClientHandler thread is created and linked with the dedicated server-side socket of the new client
3. The client handler listens and waits for a valid request from the client (START BOT, PUT row col, or QUIT).
4. In singleplayer mode, when receiving the START BOT request, a new Game object and a BotPlayer are created. If the player chooses the symbol O, the bot plays first.
5. Each time the player sends a PUT command, the handler verifies the input, updates the game grid by sending a message to the Game instance, and checks the game status (win, draw, ongoing).
6. The server sends back the updated grid and game status to the client through the socket's output stream.
7. The loop continues until the client sends a QUIT command or the game reaches a final state (X WON, O WON, or DRAW).
8. When the game is in a final state, the user has only one choice : quit the game. When he quits, the ClientHandler closes the socket and finishes its thread.

3 TCP Stream

The following method is used to read messages received through a TCP connection.

Listing 1 – readMessage()

```
public String readMessage() {
    buffer.setLength(0);
    String line = null;
    do {
        try {
            line = bufferedReader.readLine();
            if (line == null) return null; // Connection is
               closed
            else {
                buffer.append(line).append("\r\n");
            }
        } catch (SocketTimeoutException e) {
            System.out.println("Client silent too long time: " +
               e.getMessage());
            return null;
        } catch (IOException e) {
            System.out.println("Error while reading input stream:
               " + e.getMessage());
            return null;
        }
    } while (line != null && !line.isEmpty());
    return buffer.toString().replaceAll("[\\r\\n]+$", "");}
```

This method is used to read the requests from the TCP stream. Since TCP is stream-oriented, data may not arrive in complete messages but as a continuous flow of bytes. To manage this, we use a buffer to rebuild full messages following the protocol format (Tic tac toe).
First, we simply set the buffer's length to 0 to clear any previous message content as the same buffer is reused (Stringbuilder). Then, we have a do while loop that reads data up to "\n" (constituting a line). This is achieved by readLine from BufferedReader. Each line is appended with "\r\n" to return to the line (this is useful in particular when reading the grid message), and it is added to the buffer. The do while loop stops when we read a null line (which means that the connection is closed) or an empty line (the end of the message, specified in the Tic tac toe protocol). Finally, before returning what we have read, we replace all "\r" and "\n" at the end of the message so that we avoid empty lines in the returned string message.

4 Multi-thread organisation

First, we have the ClientHandler class, which extends Thread. When a new client connects, the server accepts the connection and creates a new ClientHandler, linked to this client. So, for example, if we have N clients, we have N ClientHandler threads running on the server-side, each communicating with its corresponding client. Each client has its own handler that manages its communication with the server. This allows the server to communicate simultaneously with all clients independently.

Listing 2 – Main loop of TictactoeServer

```
while (true) {
    Socket clientSocket = serveurSocket.accept();
    ClientHandler clientHandler = new ClientHandler(clientSocket,
       matchmaking);
    clientHandler.start();}
```

Each ClientHandler listens to requests from its client and processes them. This enables multiple players to play at the same time, each in a separate thread. While, the server remains free to accept new connections and create other threads.

We also synchronized the game. In fact, in multiplayer, two ClientHandler instances can access the same Game object. To prevent data corruption, every function that can be called by both handlers and that accesses shared data is marked with the synchronized keyword. This ensures that only one thread can modify or check the game state at a same time.

Listing 3 – Synchronized method in Game

```
public synchronized MoveResult playMove(int r, int c, char symbol) {
    if (finished) return MoveResult.GAME_FINISHED;
    ...}
```

For example, without the synchronized keyword, if both players send a PUT command at almost the same moment, both threads could try to modify the same cell of the grid simultaneously. In that case, one player's move could overwrite the other one, resulting an inconsistent game state. With synchronization, the first thread to enter the method locks it until it finishes execution and the second thread must wait before performing its move.

Finally, synchronization is used only where necessary and to protect shared resources (the game grid and the matchmaking).

5 Robustness

Four main rules make the server robust, as presented below :

1. **Input validation.** All client commands must be pre-validated. For example, when the user has to choose the game mode, if he does not enter "1" or "2", the program keeps asking until the input is correct. We do the same process when choosing the symbol (X or O). If the client gives too many arguments we do not take in account its input and the program specifies what we expect.
2. **Error handling.** All invalid requests based on user input, never crash the server. Instead of this, the server replies with an explicit messsage such as "WRONG", "NOT YOUR TURN" or "INVALID RANGE" et cetera. This keeps the connection open and allows the client to continue playing without restarting the program.
3. **Socket timeout.** We use a timeout on each socket to avoid accumluation of inactive threads. Even though each client runs in a separate thread, an inactive client could keep its socket open indefinitely. The timeout ensures that the connection is automatically closed and the thread released when a client remains silent for too long.
4. **Exception management.** All input/output operations are in try catch blocks. When an exception happened, the server prints an error message, closes the socket, and continues communication with other clients without interruption. For example, if there is an exception on read, the function returns null and in this case, we close only this connection, not the others.